一、实验内容

（一）MNIST数据集分类

1.加载数据（MNIST)

代码：

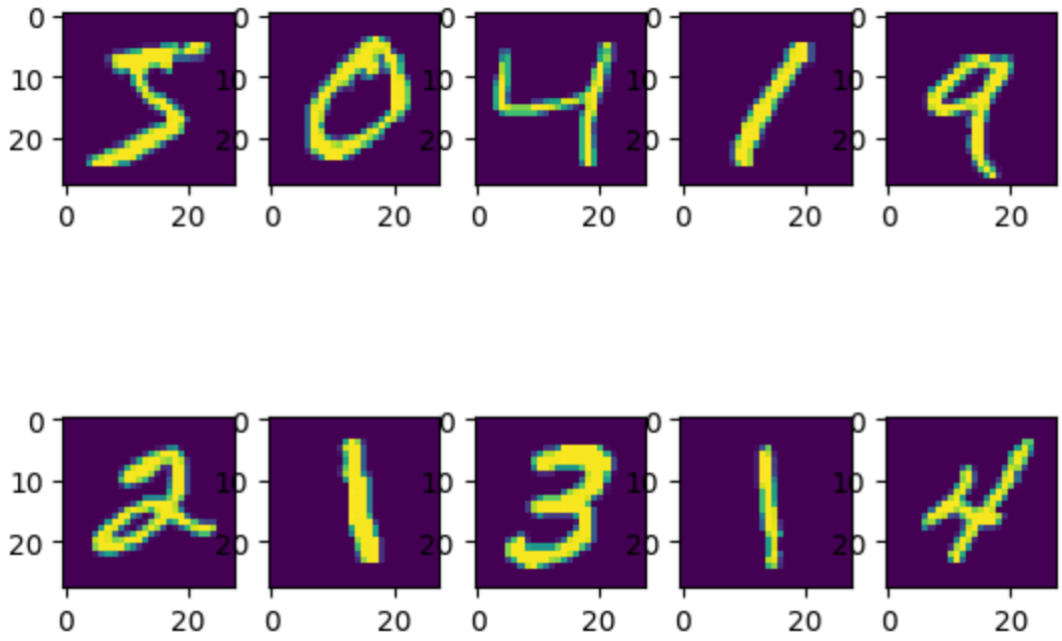
```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
# 使用GPU训练，可以在菜单 "代码执行工具" -> "更改运行时类型" 里进行设置
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
# 函数：统计模型的参数个数
def get_n_params(model):
        np = 0
        for p in list(model.parameters()):
                np += p.nelement()
        return np


input_size = 28*28      # images are 28x28 pixels
output_size = 10          # there are 10 classes
train_loader = torch.utils.data.DataLoader(
        datasets.MNIST('../data', train=True, download=True,
                                transform=transforms.Compose([
                                        transforms.ToTensor(),
                                        transforms.Normalize((0.1307,), (0.3081,))
                                ])),
        batch_size=64, shuffle=True)
test_loader = torch.utils.data.DataLoader(
        datasets.MNIST('../data', train=False, download=True,
                                transform=transforms.Compose([
                                        transforms.ToTensor(),
                                        transforms.Normalize((0.1307,), (0.3081,))
                                ])),
        batch_size=1000, shuffle=True)
```

```
plt.figure()
for i in range(10):
        plt.subplot(2, 5, i + 1)
        image, _ = train_loader.dataset.__getitem__(i)
        plt.imshow(image.squeeze().numpy())
```

运行结果：

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 106MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 4.17MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 17.1MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 12.3MB/s]
```



2.创建网络

代码：

```
class FC2Layer(nn.Module):
    def __init__(self, input_size, n_hidden, output_size):
        super(FC2Layer, self).__init__()
        self.input_size = input_size
        self.network = nn.Sequential(
            nn.Linear(input_size, n_hidden),
            nn.ReLU(),
            nn.Linear(n_hidden, n_hidden),
            nn.ReLU(),
            nn.Linear(n_hidden, output_size),
            nn.LogSoftmax(dim=1)
        )

    def forward(self, x):
        x = x.view(-1, self.input_size)
        return self.network(x)
```

```
class CNN(nn.Module):
    def __init__(self, input_size, n_feature, output_size):
        super(CNN, self).__init__()
        self.n_feature = n_feature
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=n_feature, kernel_size=5)
        self.conv2 = nn.Conv2d(n_feature, n_feature, kernel_size=5)
        self.fc1 = nn.Linear(n_feature*4*4, 50)
        self.fc2 = nn.Linear(50, 10)

    def forward(self, x, verbose=False):
        x = self.conv1(x)
        x = F.relu(x)
        x = F.max_pool2d(x, kernel_size=2)
        x = self.conv2(x)
        x = F.relu(x)
        x = F.max_pool2d(x, kernel_size=2)
        x = x.view(-1, self.n_feature*4*4)
        x = self.fc1(x)
        x = F.relu(x)
        x = self.fc2(x)
        x = F.log_softmax(x, dim=1)
        return x
```

```
class CNN(nn.Module):
    def __init__(self, input_size, n_feature, output_size):
        super(CNN, self).__init__()
        self.n_feature = n_feature
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=n_feature, kernel_size=5)
        self.conv2 = nn.Conv2d(n_feature, n_feature, kernel_size=5)
        self.fc1 = nn.Linear(n_feature*4*4, 50)
        self.fc2 = nn.Linear(50, 10)

    def forward(self, x, verbose=False):
        x = self.conv1(x)
        x = F.relu(x)
        x = F.max_pool2d(x, kernel_size=2)
        x = self.conv2(x)
        x = F.relu(x)
        x = F.max_pool2d(x, kernel_size=2)
        x = x.view(-1, self.n_feature*4*4)
        x = self.fc1(x)
        x = F.relu(x)
        x = self.fc2(x)
        x = F.log_softmax(x, dim=1)
        return x
```

```
def train(model):
    model.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = F.nll_loss(output, target)
        loss.backward()
        optimizer.step()
        if batch_idx % 100 == 0:
            print('Train: [{}/{} ({:.0f}%)]\tLoss: {:.6f}'.format(
                batch_idx * len(data), len(train_loader.dataset),
                100. * batch_idx / len(train_loader), loss.item()))
                        100. * batch_idx / len(train_loader), loss.item()))
def test(model):
    model.eval()
    test_loss = 0
    correct = 0
    for data, target in test_loader:
        data, target = data.to(device), target.to(device)
        output = model(data)
        test_loss += F.nll_loss(output, target, reduction='sum').item()
        pred = output.data.max(1, keepdim=True)[1]
        correct += pred.eq(target.data.view_as(pred)).cpu().sum().item()
    test_loss /= len(test_loader.dataset)
    accuracy = 100. * correct / len(test_loader.dataset)
    print('\nTest set: Average loss: {:.4f}, Accuracy: {}/{} ({:.0f}%)\n'.format(
        test_loss, correct, len(test_loader.dataset),
        accuracy))
```

3.在小型全连接网络上训练

代码:

```
n_hidden = 8 # number of hidden units

model_fnn = FC2Layer(input_size, n_hidden, output_size)
model_fnn.to(device)
optimizer = optim.SGD(model_fnn.parameters(), lr=0.01, momentum=0.5)
print('Number of parameters: {}'.format(get_n_params(model_fnn)))

train(model_fnn)
test(model_fnn)
```

运行结果：

```
Number of parameters: 6442
Train: [0/60000 (0%)]   Loss: 2.352954
Train: [6400/60000 (11%)]       Loss: 2.072363
Train: [12800/60000 (21%)]      Loss: 1.681930
Train: [19200/60000 (32%)]      Loss: 1.183268
Train: [25600/60000 (43%)]      Loss: 0.754256
Train: [32000/60000 (53%)]      Loss: 0.554821
Train: [38400/60000 (64%)]      Loss: 0.553832
Train: [44800/60000 (75%)]      Loss: 0.471356
Train: [51200/60000 (85%)]      Loss: 0.548254
Train: [57600/60000 (96%)]      Loss: 0.424918

Test set: Average loss: 0.4553, Accuracy: 8664/10000 (87%)
```

代码：

```
# number of feature maps
n_features = 6

model_cnn = CNN(input_size, n_features, output_size)
model_cnn.to(device)
optimizer = optim.SGD(model_cnn.parameters(), lr=0.01, momentum=0.5)
print('Number of parameters: {}'.format(get_n_params(model_cnn)))

train(model_cnn)
test(model_cnn)
```

运行结果：

```
Number of parameters: 6422
Train: [0/60000 (0%)]   Loss: 2.306418
Train: [6400/60000 (11%)]       Loss: 1.963346
Train: [12800/60000 (21%)]      Loss: 0.485415
Train: [19200/60000 (32%)]      Loss: 0.309947
Train: [25600/60000 (43%)]      Loss: 0.202978
Train: [32000/60000 (53%)]      Loss: 0.304451
Train: [38400/60000 (64%)]      Loss: 0.192423
Train: [44800/60000 (75%)]      Loss: 0.116544
Train: [51200/60000 (85%)]      Loss: 0.093352
Train: [57600/60000 (96%)]      Loss: 0.255474

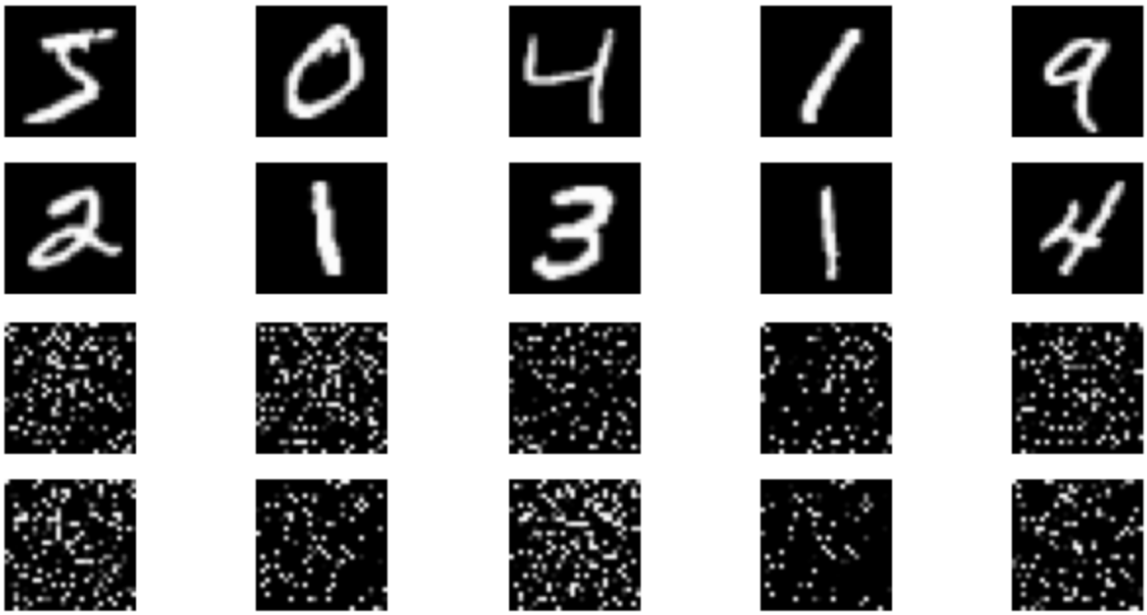
Test set: Average loss: 0.1727, Accuracy: 9460/10000 (95%)
```

4.打乱像素顺序再次在两个网络上训练与测试

代码：

```
#  这里解释一下  torch.randperm  函数，
#  给定参数n，返回一个从0到n-1的随机整数排列
perm  =  torch.randperm(784)
plt.figure(figsize=(8,  4))
for  i  in  range(10):
        image,  _  =  train_loader.dataset.__getitem__(i)
        #  permute  pixels
        image_perm  =  image.view(-1,  28*28).clone()
        image_perm  =  image_perm[:,  perm]
        image_perm  =  image_perm.view(-1,  1,  28,  28)
        plt.subplot(4,  5,  i  +  1)
        plt.imshow(image.squeeze().numpy(),  'gray')
        plt.axis('off')
        plt.subplot(4,  5,  i  +  11)
        plt.imshow(image_perm.squeeze().numpy(),  'gray')
        plt.axis('off')
```

运行结果：



代码：

```
# 对每个 batch 里的数据，打乱像素顺序的函数
def perm_pixel(data, perm):
    # 转化为二维矩阵
    data_new = data.view(-1, 28*28)
    # 打乱像素顺序
    data_new = data_new[:, perm]
    # 恢复为原来4维的 tensor
    data_new = data_new.view(-1, 1, 28, 28)
    return data_new

# 训练函数
def train_perm(model, perm):
    model.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        # 像素打乱顺序
        data = perm_pixel(data, perm)
        optimizer.zero_grad()
        output = model(data)
        loss = F.nll_loss(output, target)
        loss.backward()
        optimizer.step()
        if batch_idx % 100 == 0:
            print('Train: [{}/{} ({:.0f}%)]\tLoss: {:.6f}'.format(
                batch_idx * len(data), len(train_loader.dataset),
                100. * batch_idx / len(train_loader), loss.item()))
```

```
# 测试函数
def test_perm(model, perm):
    model.eval()
    test_loss = 0
    correct = 0
    for data, target in test_loader:
        data, target = data.to(device), target.to(device)
        # 像素打乱顺序
        data = perm_pixel(data, perm)
        output = model(data)
        test_loss += F.nll_loss(output, target, reduction='sum').item()
        pred = output.data.max(1, keepdim=True)[1]
        correct += pred.eq(target.data.view_as(pred)).cpu().sum().item()
    test_loss /= len(test_loader.dataset)
    accuracy = 100. * correct / len(test_loader.dataset)
    print('\nTest set: Average loss: {:.4f}, Accuracy: {}/{} ({:.0f}%)\n'.format(
        test_loss, correct, len(test_loader.dataset),
        accuracy))
```

代码:

```
perm = torch.randperm(784)
n_hidden = 8 # number of hidden units
model_fnn = FC2Layer(input_size, n_hidden, output_size)
model_fnn.to(device)
optimizer = optim.SGD(model_fnn.parameters(), lr=0.01, momentum=0.5)
print('Number of parameters: {}'.format(get_n_params(model_fnn)))

train_perm(model_fnn, perm)
test_perm(model_fnn, perm)
```

运行结果：

```
Number of parameters: 6442
Train: [0/60000 (0%)]   Loss: 2.377473
Train: [6400/60000 (11%)]       Loss: 1.988394
Train: [12800/60000 (21%)]      Loss: 1.328150
Train: [19200/60000 (32%)]      Loss: 1.025844
Train: [25600/60000 (43%)]      Loss: 0.658906
Train: [32000/60000 (53%)]      Loss: 0.797518
Train: [38400/60000 (64%)]      Loss: 0.460844
Train: [44800/60000 (75%)]      Loss: 0.606015
Train: [51200/60000 (85%)]      Loss: 0.735450
Train: [57600/60000 (96%)]      Loss: 0.599517

Test set: Average loss: 0.4444, Accuracy: 8658/10000 (87%)
```

代码：

```
perm = torch.randperm(784)
# number of feature maps
n_features = 6

model_cnn = CNN(input_size, n_features, output_size)
model_cnn.to(device)
optimizer = optim.SGD(model_cnn.parameters(), lr=0.01, momentum=0.5)
print('Number of parameters: {}'.format(get_n_params(model_cnn)))

train_perm(model_cnn, perm)
test_perm(model_cnn, perm)
```

运行结果：

```
Number of parameters: 6422
Train: [0/60000 (0%)]   Loss: 2.310130
Train: [6400/60000 (11%)]       Loss: 2.275913
Train: [12800/60000 (21%)]      Loss: 2.168187
Train: [19200/60000 (32%)]      Loss: 1.880414
Train: [25600/60000 (43%)]      Loss: 1.237270
Train: [32000/60000 (53%)]      Loss: 0.932580
Train: [38400/60000 (64%)]      Loss: 0.850469
Train: [44800/60000 (75%)]      Loss: 0.796021
Train: [51200/60000 (85%)]      Loss: 0.727062
Train: [57600/60000 (96%)]      Loss: 0.473222

Test set: Average loss: 0.5804, Accuracy: 8208/10000 (82%)
```

从打乱像素顺序的实验结果来看，全连接网络的性能基本上没有发生变化。但是卷积神经网络的性能明显下降。

（二）CIFAR10 数据集分类

1.导入依赖与数据集加载

代码：

```
import torch
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
# 使用GPU训练，可以在菜单 "代码执行工具" -> "更改运行时类型" 里进行设置
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

# 注意下面代码中：训练的 shuffle 是 True，测试的 shuffle 是 false
# 训练时可以打乱顺序增加多样性，测试是没有必要
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)

trainloader = torch.utils.data.DataLoader(trainset, batch_size=64,
                                          shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)

testloader = torch.utils.data.DataLoader(testset, batch_size=8,
                                         shuffle=False, num_workers=2)

classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
```

运行结果：

```
100%|██████████| 170M/170M [58:54<00:00, 48.2kB/s]
```

2.数据集图像可视化

代码：

```
def imshow(img):
    plt.figure(figsize=(8,8))
    # 转换到 [0,1] 之间
    img = img / 2 + 0.5
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()

# 得到一组图像
images, labels = iter(trainloader).next()
# 展示图像
imshow(torchvision.utils.make_grid(images))
# 展示第一行图像的标签
for j in range(8):
    print(classes[labels[j]])
```

运行结果：

...




```
dog
bird
cat
bird
dog
ship
plane
truck
```

3.定义网络、损失函数与优化器

代码:

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

# 网络放到GPU上
net = Net().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters(), lr=0.001)
```

4.训练网络

代码:

```
for epoch in range(10):
    # 重复多轮训练
    for i, (inputs, labels) in enumerate(trainloader):
        inputs = inputs.to(device)
        labels = labels.to(device)
        # 优化器梯度归零
        optimizer.zero_grad()
        # 正向传播 + 反向传播 + 优化
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        # 输出统计信息
        if i % 100 == 0:
            print('Epoch: %d Minibatch: %5d loss: %.3f' %(epoch + 1, i + 1, loss.item()))
print('Finished Training')
```

运行结果：

```
Epoch: 1 Minibatch:     1 loss: 2.292
Epoch: 1 Minibatch:   101 loss: 1.797
Epoch: 1 Minibatch:   201 loss: 1.786
Epoch: 1 Minibatch:   301 loss: 1.672
Epoch: 1 Minibatch:   401 loss: 1.373
Epoch: 1 Minibatch:   501 loss: 1.439
Epoch: 1 Minibatch:   601 loss: 1.314
Epoch: 1 Minibatch:   701 loss: 1.423
Epoch: 2 Minibatch:     1 loss: 1.517
Epoch: 2 Minibatch:   101 loss: 1.420
Epoch: 2 Minibatch:   201 loss: 1.425
Epoch: 2 Minibatch:   301 loss: 1.011
Epoch: 2 Minibatch:   401 loss: 1.345
Epoch: 2 Minibatch:   501 loss: 1.112
Epoch: 2 Minibatch:   601 loss: 1.359
Epoch: 2 Minibatch:   701 loss: 1.166
Epoch: 3 Minibatch:     1 loss: 1.273
Epoch: 3 Minibatch:   101 loss: 1.274
Epoch: 3 Minibatch:   201 loss: 1.326
Epoch: 3 Minibatch:   301 loss: 1.179
Epoch: 3 Minibatch:   401 loss: 1.170
Epoch: 3 Minibatch:   501 loss: 1.253
Epoch: 3 Minibatch:   601 loss: 1.159
Epoch: 3 Minibatch:   701 loss: 1.384
Epoch: 4 Minibatch:     1 loss: 1.364
```

5.测试集单批样本预测

代码：

```
# 得到一组图像
images, labels = iter(testloader).next()
# 展示图像
imshow(torchvision.utils.make_grid(images))
# 展示图像的标签
for j in range(8):
    print(classes[labels[j]])
```

```
outputs = net(images.to(device))
_, predicted = torch.max(outputs, 1)

# 展示预测的结果
for j in range(8):
        print(classes[predicted[j]])
```

运行结果：

```
cat
frog
dog
cat
frog
ship
plane
truck
```

6.全测试集准确率评估

代码：

```
correct = 0
total = 0
for data in testloader:
        images, labels = data
        images, labels = images.to(device), labels.to(device)
        outputs = net(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
print('Accuracy of the network on the 10000 test images: %d %%' % (100 * correct / total))
```

运行结果：

```
Accuracy of the network on the 10000 test images: 61 %
```

（三）VGG16对CIFAR10分类

1.定义 dataloader

代码：

```
import torch
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
# 使用GPU训练，可以在菜单 "代码执行工具" -> "更改运行时类型" 里进行设置
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
transform_train = transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010))])
transform_test = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010))])
trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform_train)
testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform_test)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=128, shuffle=True, num_workers=2)
testloader = torch.utils.data.DataLoader(testset, batch_size=128, shuffle=False, num_workers=2)
classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
```

运行结果：

```
100%|██████████| 170M/170M [59:20<00:00, 47.9kB/s]
```

2.VGG 网络定义

代码：

```
class VGG(nn.Module):
    def __init__(self):
        super(VGG, self).__init__()
        self.cfg = [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M']
        self.features = self._make_layers(cfg)
        self.classifier = nn.Linear(2048, 10)

    def forward(self, x):
        out = self.features(x)
        out = out.view(out.size(0), -1)
        out = self.classifier(out)
        return out

    def _make_layers(self, cfg):
        layers = []
        in_channels = 3
        for x in cfg:
            if x == 'M':
                layers += [nn.MaxPool2d(kernel_size=2, stride=2)]
            else:
                layers += [nn.Conv2d(in_channels, x, kernel_size=3, padding=1),
                           nn.BatchNorm2d(x),
                           nn.ReLU(inplace=True)]
                in_channels = x
        layers += [nn.AvgPool2d(kernel_size=1, stride=1)]
        return nn.Sequential(*layers)

# 网络放到GPU上
net = VGG().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters(), lr=0.001)
```

3.网络训练

代码：

```
for epoch in range(10): # 重复多轮训练
    for i, (inputs, labels) in enumerate(trainloader):
        inputs = inputs.to(device)
        labels = labels.to(device)
        # 优化器梯度归零
        optimizer.zero_grad()
        # 正向传播 + 反向传播 + 优化
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        # 输出统计信息
        if i % 100 == 0:
            print('Epoch: %d Minibatch: %5d loss: %.3f' %(epoch + 1, i + 1, loss.item()))
print('Finished Training')
```

4.测试验证准确率

代码：

```
correct = 0
total = 0
for data in testloader:
    images, labels = data
    images, labels = images.to(device), labels.to(device)
    outputs = net(images)
    _, predicted = torch.max(outputs.data, 1)
    total += labels.size(0)
    correct += (predicted == labels).sum().item()
print('Accuracy of the network on the 10000 test images: %.2f %%' % (
    100 * correct / total))
```

（四）思考总结

通过这三组递进的卷积神经网络实验，我从基础的 MNIST 手写数字分类入手，逐步掌握了卷积、池化层的核心作用与 CNN 的基本搭建方法，验证了卷积网络相比全连接网络更擅长挖掘图像的局部空间特征，也通过像素打乱实验直观理解了 CNN 依赖图像像素空间相关性的归纳偏置；在此基础上完成的 CIFAR-10 彩色图像分类实验，让我熟悉了 LeNet 风格的经典卷积网络结构与多通道图像的处理逻辑，而后续的简化 VGG 网络实验则直观展现了网络深度提升、数据增强、批量归一化等策略对分类准确率的显著提升作用，从基础网络的 64% 准确率提升至 84.92%，也让我对深度学习中模型结构设计、数据预处理与训练优化共同决定模型性能的逻辑有了更完整的认知。

二、问题总结与体会

1.dataloader 里面 shuffle 取不同值有什么区别?

dataloader 里 shuffle 设置为 True 时，每个 epoch 开始前会打乱数据集样本顺序，避免模型按固定样本顺序学习产生过拟合；shuffle 为 False 则保持原始顺序，一般测试集不开启打乱，保证结果可复现。

2.transform 里，取了不同值，这个有什么区别?

ransform 是图像预处理工具，不同参数代表不同操作，比如随机裁剪、翻转、归一化，训练时用随机变换可以扩充数据、增强泛化能力，测试阶段只用固定归一化，防止引入随机噪声干扰预测结果。

3.epoch 和 batch 的区别?

batch 指一次送入网络训练的样本数量，epoch 代表完整遍历一遍全部训练数据集，一个 epoch 会包含多个 batch，batch 决定单次梯度更新的数据量，epoch 控制整体数据集迭代轮数。

4.1x1的卷积和 FC 有什么区别？主要起什么作用?

1×1 卷积可以在不改变特征图长宽的情况下调整通道数量，保留空间信息，而全连接层会把二维特征展平，丢失空间位置信息，1×1 卷积常用于降维、跨通道信息融合，全连接多用于最后分类输出。

5.residual leanring 为什么能够提升准确率?

残差学习通过残差跳跃连接，让网络学习输入与输出之间的残差而不是直接学习映射，缓解深层网络梯度消失问题，使得深层模型更容易收敛，从而提升模型准确率。

6.代码练习二里，网络和1989年 Lecun 提出的 LeNet 有什么区别?

实验 4 所用 LeNet 基于 PyTorch 实现，使用 ReLU 激活函数，搭配现代优化器，原始 1989 版 LeNet 使用 sigmoid 激活、平均池化，硬件和训练策略都比较老旧，网络细节与训练方式存在明显差别。

7.代码练习二里，卷积以后feature map 尺寸会变小，如何应用 Residual Learning?

当卷积操作改变特征图通道或尺寸无法直接相加时，残差连接可以用 1×1 卷积对旁路输入做变换，调整它的通道数和尺寸，让主干分支和旁路分支维度匹配后再相加，实现残差学习。

8.有什么方法可以进一步提升准确率?

想要提升模型准确率，可以增加数据集、使用数据增强，适当调整网络深度，选择合适的优化器与学习率策略，加入正则化方法抑制过拟合，同时调整 batch 大小和训练轮数。